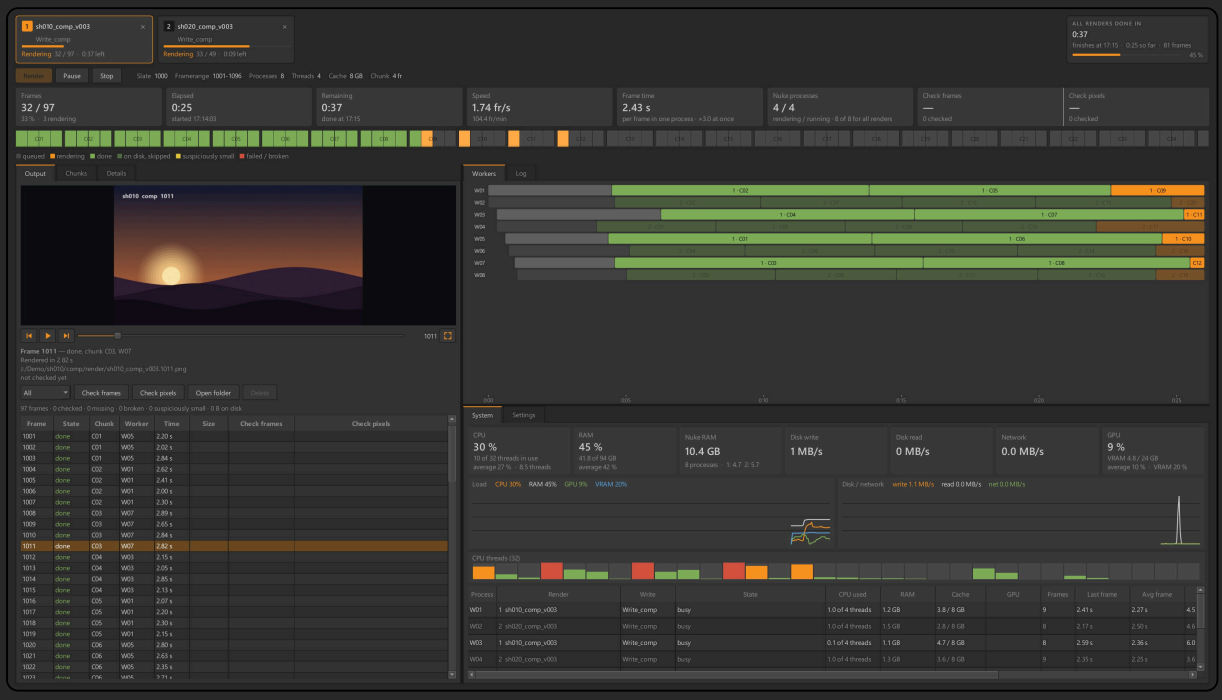
GUÍA DE USO · VERSIÓN 0.99.6

JKrender

Render local rápido para Nuke. El render se divide en trozos (chunks) que varios procesos de Nuke calculan a la vez. Tras el render, JKrender revisa cada frame en el disco.

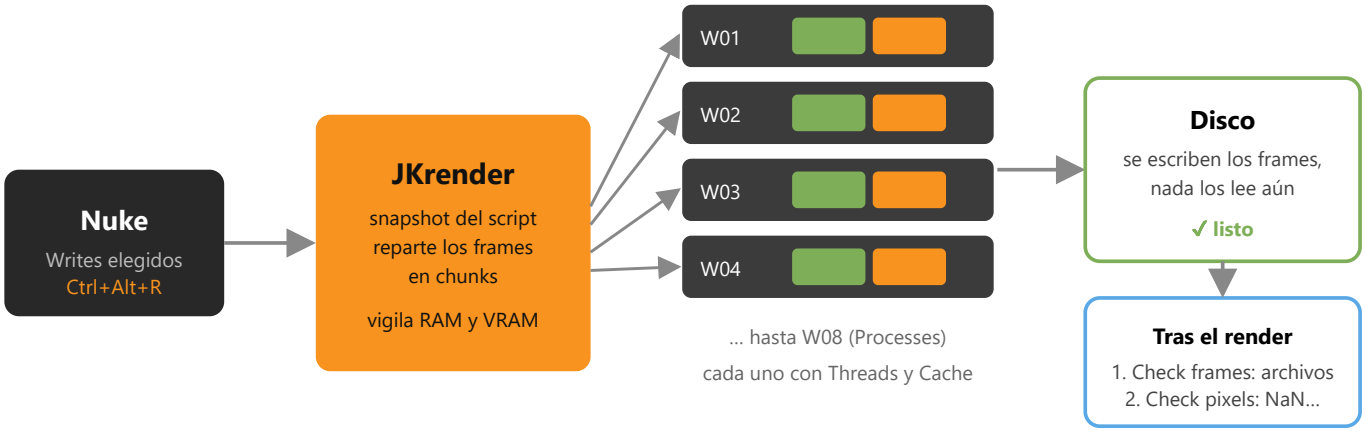


Índice

1 Cómo funciona JKrender
2 Instalación y arranque desde Nuke
3 Mapa de la ventana
4 Tarjetas de render y fila de control
5 Mosaicos y mapa de frames
6 Panel Output: vista previa y frames
7 Paneles Chunks y Details
8 Paneles Workers y Log
9 Panel System
10 Settings: cada ajuste
11 Cómo ajustar un render
12 Test this script
13 Vigilancia de RAM y VRAM
14 Comprobaciones, sonidos, caídas, datos
15 Ratón y teclas: chuleta
16 Cuando algo no va

1 Cómo funciona JKrender

Un render normal de Nuke corre en un solo proceso que calcula todo el script frame a frame. Pero un Nuke rara vez usa los 32 hilos de la CPU. Por eso JKrender lanza **varios procesos de Nuke a la vez**. Cada uno **abre el script una sola vez** y luego recibe trozos de trabajo, llamados **chunks**: por ejemplo 10 frames seguidos.



Hacia el final los chunks se hacen más pequeños, así todos los procesos terminan casi a la vez.

Tres números que deciden la velocidad

Processes

Cuántos procesos de Nuke corren a la vez (para todos los renders juntos). Más procesos hacen más trabajo a la vez, pero también usan más RAM y VRAM.

Threads

Cuántos hilos de CPU recibe cada proceso (Nuke `-m`). **Processes × Threads** debería dar más o menos los hilos del PC (aquí 32).

Cache

El límite de caché de un proceso en GB (Nuke `-c`). En realidad un proceso usa más. Cuánto, lo muestra System › RAM.

> Por defecto son **8 procesos × 4 hilos**. Los scripts de comp pesados que un solo Nuke ya llena (Defocus, Denoise) suelen ir más rápido en **4 × 8** y usan la mitad de RAM. Los scripts cuya VRAM crece quieren menos procesos. Qué es lo mejor para tu script lo mide **Test this script** (capítulo 12).

Dónde se guarda cada cosa

Qué	Dónde
Ajustes	`~/.nuke/JKrender/settings/settings.json` (cada usuario tiene los suyos). Las versiones antiguas los tenían en `~/.jkrender`; JKrender los mueve solo en su primer arranque.
Logs de render	`~/.nuke/JKrender/log/`: una carpeta por render (snapshot del script, job, el log de cada proceso de Nuke y `report.json`). Los de más de 15 días se borran.
Resultados de tests	`~/.nuke/JKrender/settings/test_results`
Frames renderizados	Donde los manda el Write. JKrender nunca los borra solo, únicamente con el botón Delete.

2 Instalación y arranque desde Nuke

Instalación (una vez)

1. Copia la carpeta entera `JKrender` en `%USERPROFILE%\.nuke\`. Descargada de GitHub como zip se llama `JKrender-main`: renómbrala a `JKrender`.

2. Añade esta línea a `~/.nuke/init.py` (crea el archivo si no existe):

```
nuke.pluginAddPath("./JKrender")
```

3. Reinicia Nuke. En el primer arranque, JKrender pregunta si descargar numpy y OpenEXR para la vista previa EXR (unos 15 MB). Si dices que no, el render funciona igual, solo sin vista previa EXR. Para que vuelva a preguntar, borra el archivo `lib/DECLINED`.

Opcional: para los detalles y el reproductor de MOV/MP4, pon ffprobe/ffmpeg en el PATH (`winget install ffmpeg`). Hace falta Nuke 16 (PySide6).

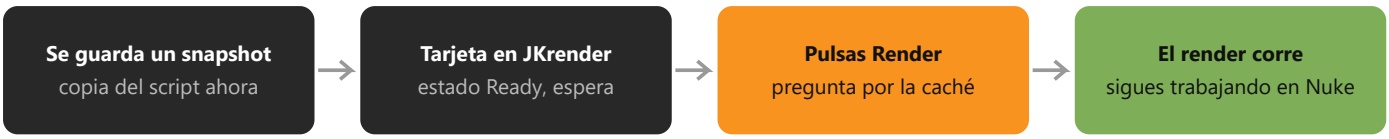
Nueva versión: copia los archivos nuevos sobre los viejos. Las carpetas `lib`, `log` y `settings` se quedan.

El menú JKrender en Nuke

Opción	Qué hace
Render Selected Write(s) `Ctrl+Alt+R`	Envía los Writes elegidos (o Groups con Writes) a JKrender. Cada Write tiene su propia tarjeta. Si la ventana aún no está abierta, se abre.
Open Renders Folder	Abre la carpeta con los logs de render.

Si el script no está guardado, pregunta *Save it and render?*: se renderiza la versión guardada.

Qué pasa al enviarlo

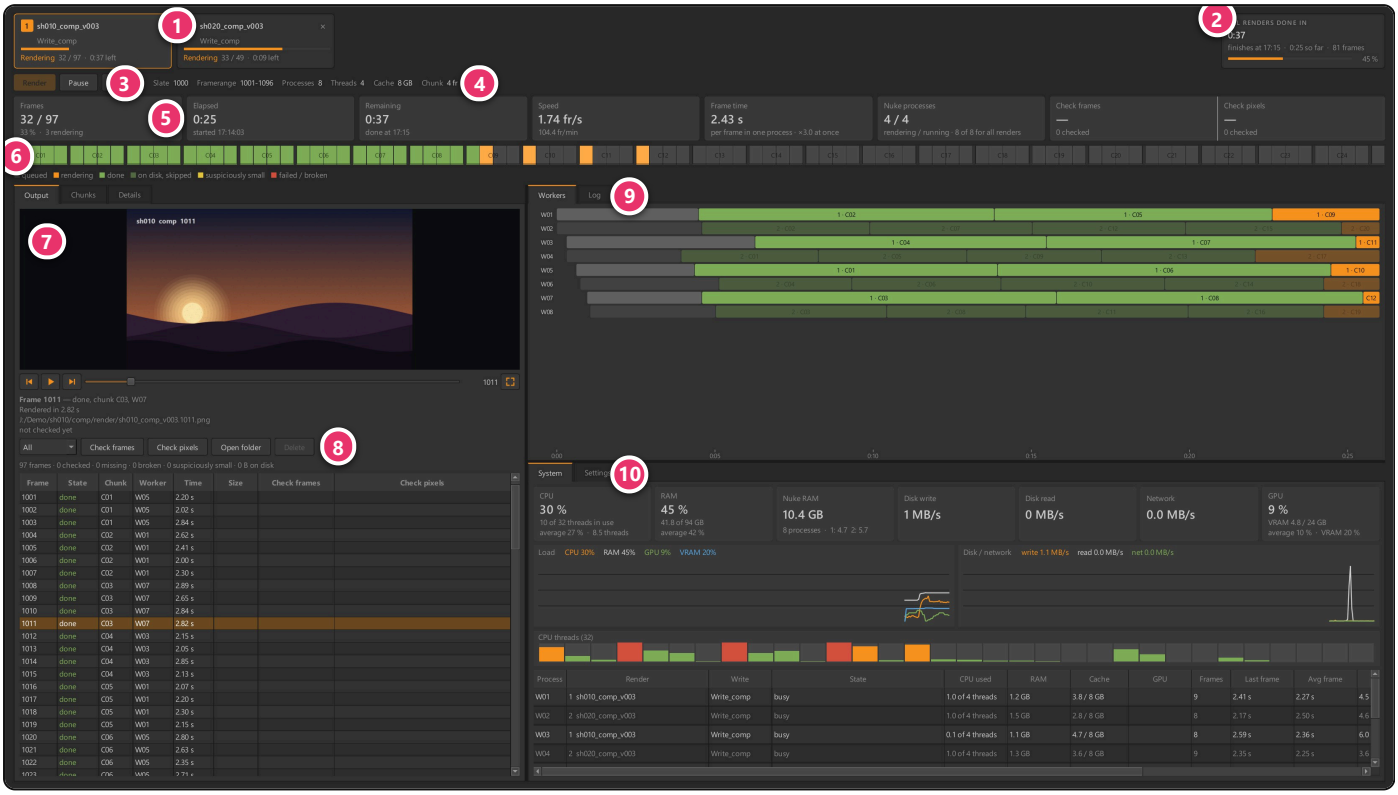


> **Snapshot:** se renderiza el script tal como estaba al enviarlo. Puedes seguir trabajando y guardando en Nuke enseguida; el render en marcha no se entera. Para renderizar un script cambiado, envíalo otra vez desde Nuke: el envío más nuevo sustituye a la tarjeta vieja.

> **Pregunta por la caché:** si hay un Nuke abierto, Render pregunta si vaciar sus buffers, la caché de reproducción y la caché Blink, para que el render tenga más memoria. La respuesta por defecto es **No** y el render arranca en ambos casos. La caché de disco no se toca. Tras actualizar JKrender, reinicia Nuke una vez para que funcione.

3 Mapa de la ventana

Arriba están las tarjetas de render y la fila de control, debajo los números y el mapa de frames. El resto son tres paneles: a la izquierda **Output / Chunks / Details**, arriba a la derecha **Workers / Log** y abajo a la derecha **System / Settings**. Los bordes entre paneles se pueden arrastrar.



1. Tarjetas de render: un Write (render) por tarjeta. Un clic lo elige.
2. Cuándo deberían estar listos todos los renders.
3. Render, Pause, Stop del render elegido.
4. Slate, Framerange, Processes, Threads, Cache y Chunk de este render (solo lectura).
5. Mosaicos con números: frames, tiempo, velocidad, procesos, comprobaciones.
6. El mapa de frames por chunks. Un clic en un frame lo muestra.
7. Output: vista previa de un frame con reproducción.
8. Output: filtro, comprobaciones, carpeta, borrar y la lista de frames.
9. Workers: línea de tiempo de cada proceso de Nuke.
10. System: carga del PC y tabla de procesos.

4 Tarjetas de render y fila de control

Una tarjeta de render

Cada Write enviado es una tarjeta: número, nombre del script, nombre del Write, barra de progreso y estado (Ready, Waiting, Rendering, Paused, Checking output, Done, Failed, Stopped). Un **clic** elige la tarjeta y toda la ventana muestra ese render. **Shift+clic** elige varias tarjetas; Render, Pause y Stop valen entonces para todas, y Render again también para las terminadas (pregunta una vez para todas). La × cierra el render; los frames en el disco se quedan. **Tarjeta roja**: el render tiene un error (falló, Nuke no renderizó un frame, frames rotos, un error de Nuke, un proceso se cayó), también mientras renderiza; el tooltip de la tarjeta dice cuál. Un proceso cerrado por RAM o VRAM no es un error.

Los renders comparten un presupuesto de procesos. A la vez corren como mucho los que diga Settings *Max renders at once* (por defecto 3); los procesos se reparten entre ellos (8 → 4 + 4 → 3 + 3 + 2).

La tarjeta de la derecha: ALL RENDERS DONE IN

Cuándo deberían terminar todos los renders que corren o esperan, a la velocidad a la que renderiza ahora el PC. Debajo: a qué hora acaban, cuánto llevan y cuántos frames faltan. Cuando todos terminan, la tarjeta muestra **ALL RENDERS TOOK**: cuánto duró todo.

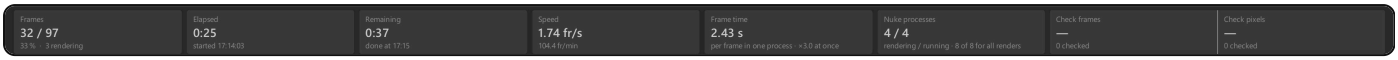
La fila de control

Elemento	Qué hace
Render	Arranca el render elegido (o varios elegidos). Salta los frames que ya están bien en el disco. Tras Stop sigue donde lo dejó (Resume). Si el render tiene frames en el disco (terminado, parado o con frames de antes), pregunta: **Only missing** renderiza solo los frames que faltan o están rotos (los demás salen verde oscuro en el mapa), **All frames again** lo renderiza todo de nuevo, **Cancel** nada.
Pause	Congela todos los procesos de Nuke del render a mitad de frame (la CPU queda libre, la memoria sigue ocupada). Otro clic continúa.
Stop	Para el render, cierra sus procesos y borra frames a medio escribir y archivos temporales de Nuke. Los frames terminados se quedan.
Slate	El número del slate: el frame antes del Framerange, cuando el slate está activado (Settings › Job).
Framerange	Los frames de este render: el rango de Nuke, salvo que lo cambies en Settings › Job.
Processes · Threads · Cache · Chunk	Con qué corre el render. Se cambian en Settings › Processes and Chunks; el número de procesos también en marcha, con clic derecho en System.
Barra de color	El resultado del render, ver abajo.

La barra de color con el resultado

Color	Qué significa	Ejemplo
Verde	Terminado, todos los frames están en el disco y pasan la comprobación.	Done in 2:06. All 97 frames are on disk and pass the check.
Amarillo	Terminado, pero algo no está verificado o parece sospechoso (frames sin comprobar, sospechosamente pequeños; un slate no cuenta como pequeño).	frames not checked · 2 suspiciously small
Rojo	Un error: el render falló, frames rotos, o Check pixels encontró un problema (NaN, un frame negro…).	2 broken frames · Output: Problems shows them
Gris	En curso: *Checking pixels…*, esperando procesos libres y similares.	Waiting: 1 render already uses the PC

5 Mosaicos y mapa de frames



Mosaico	Qué muestra
Frames	Frames hechos / todos del render, porcentaje y cuántos se renderizan ahora.
Elapsed	Cuánto lleva el render (sin pausas).
Remaining	El tiempo restante a la velocidad actual.
Speed	Frames por segundo y por minuto, todos los procesos juntos. El mejor número para comparar ajustes.
Frame time	Cuánto tarda un frame en un proceso y cuántos se calculan a la vez (× N at once).
Nuke processes	Cuántos procesos renderizan / corren, más los que cargan o esperan.
Check frames	La comprobación de archivos: frames que faltan, vacíos, cortados, ilegibles y sospechosamente pequeños.
Check pixels	La comprobación de píxeles (EXR, PNG, JPEG, TIFF, DPX): frames negros, vacíos y de un solo color, frames iguales a otro y un frame cuyo tamaño, canales o profundidad de bits difieren del resto. En EXR también NaN, inf y compresión.

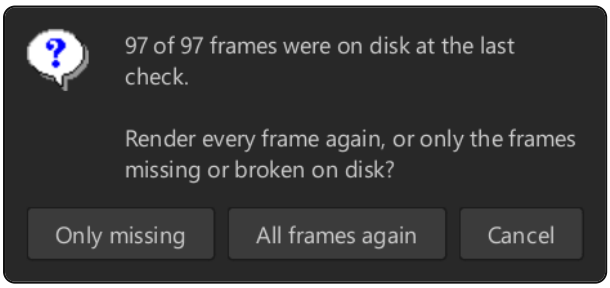
El mapa de frames



Cada cuadradito es un frame, agrupados por chunks (C01, C02…). Un **clic en un frame** lo abre en la vista previa. Colores:

en espera (queued) renderizando hecho ya estaba en el disco, saltado (Only missing) sospechosamente pequeño fallido / roto

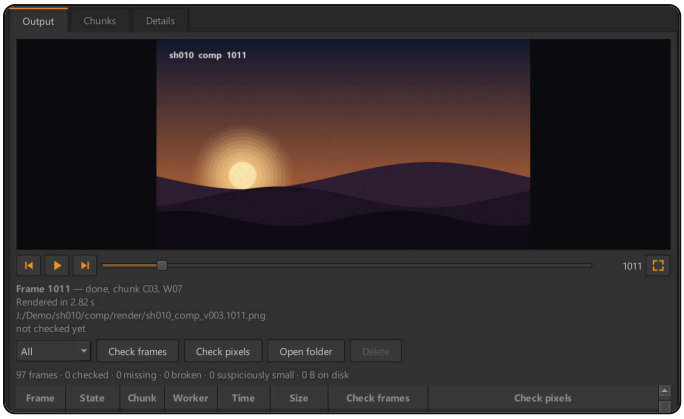
Render again: todo, o solo lo que falta



Si el render tiene frames en el disco y pulsas **Render again**, pregunta: **Only missing** (por defecto) renderiza solo los frames que faltan o están rotos; los demás se saltan y salen **verde oscuro** en el mapa. **All frames again** lo renderiza todo de nuevo, **Cancel** nada. Una película (MOV, MP4) no pregunta: es un solo archivo y se renderiza entera.

Con varias tarjetas elegidas con **Shift** pregunta una vez para todas (p. ej. *27 of 27 frames of 3 renders*…). Los frames que ya estaban en el disco al abrir el render, o de la pasada anterior al Stop, salen verde oscuro. En Output el filtro **Rendered** muestra entonces solo los frames de esta pasada, **All** todos.

6 Panel Output: vista previa y frames



Vista previa

- **Rueda del ratón**: zoom, **arrastrar** mueve la imagen.
- **Doble clic**: vuelve al frame entero. Si ya lo está, la vista crece sobre los tres paneles; se vuelve con `Esc`.
- Con zoom se carga la resolución completa (también EXR y 4K).
- `Space` arranca y para la reproducción. Nunca pulsa un botón, así que nunca para el render.

La barra bajo la vista previa

Frame anterior · **Play/Pause** (reproduce en bucle los frames terminados a los fps del script) · frame siguiente · deslizador por todos los frames · número de frame · **ampliar** (vista sobre tres paneles, `Esc` para volver). Bajo la barra: ruta, tamaño, resolución, canales y resultados de las comprobaciones del frame.

Botones

Elemento	Qué hace
Filtro	**All**: todos los frames a la vez: renderizados, saltados, rotos, *not on disk* y los que faltan por renderizar, y frames en el disco fuera del Framerange. Los demás filtros son partes de este: **Rendered**: solo los frames renderizados en esta pasada. **Problems**: errores, frames sospechosamente pequeños y lo que encontró la comprobación de píxeles. **Framerange**: solo los frames del rango elegido.
Check frames	Vuelve a revisar los archivos de todos los frames: existe, no está vacío, está entero y legible y no es mucho más pequeño que el resto. Un frame que aún no está en el disco sigue como *not on disk* y no cuenta como roto.
Check pixels	Revisa en segundo plano los píxeles de cada frame (EXR, PNG, JPEG, TIFF, DPX): frames negros, vacíos y de un solo color, frames iguales a otro en cualquier punto de la secuencia, tamaño, canales o profundidad de bits distintos; en EXR también NaN, inf y compresión. Unos 0,1 s por frame 4K. Solo avisa, no vuelve a renderizar nada.
Open folder	Abre la carpeta de salida.
Delete	Borra del disco los frames renderizados: todos (por defecto) o un rango que indiques. Pregunta antes.

La lista de frames

Columnas: **Frame**, **State**, **Chunk**, **Worker** (qué proceso lo calculó), **Time**, **Size**, **Check frames**, **Check pixels**. Con el filtro All todos los frames están en la lista desde el principio y su estado cambia mientras se renderizan. Tras **Render again** los frames no desaparecen: quedan como *on disk, queued again* con su tamaño y tiempo anteriores (se pueden ver y reproducir) hasta que el nuevo render los sobrescribe. Los frames en el disco fuera del Framerange elegido siguen en la lista con la nota *not in Framerange*.

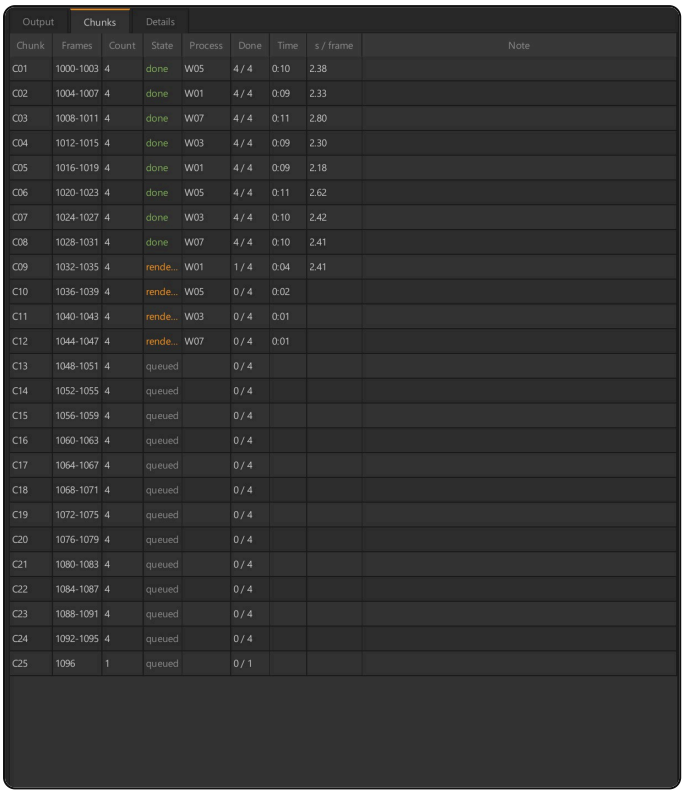


Clic derecho en un frame (o en varios elegidos):
Render this frame / batch: vuelve a renderizar solo esos frames (si el render corre, se añaden a él).
Delete this frame / batch: los borra del disco (solo si el render no corre).

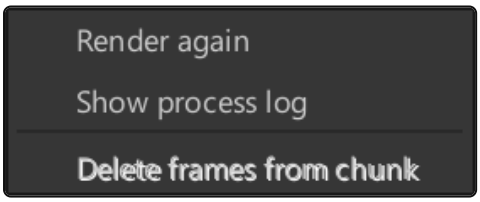
> **MOV / MP4 / MXF:** en lugar de la lista de frames aparece un archivo de película con sus detalles (códec, perfil, fps, resolución, timecode, número de frames frente a los ajustes del Write) y un reproductor con lupa. Botones Check video, Open folder, Delete video. Una película no se puede dividir, así que la renderiza un proceso con todos los hilos; Settings lo muestran así también (Processes 1, bloqueado). Los ajustes de los demás renders no cambian.

7 Paneles Chunks y Details

Chunks



Una fila por chunk: **Chunk** (C01…), **Frames** (rango), **Count**, **State**, **Process** (quién lo calcula), **Done**, **Time**, **s / frame** y **Note** (por qué se repitió un chunk y similares).



Clic derecho: Render again (el chunk de nuevo), **Show process log** (el log del proceso que lo calculó), **Delete frames from chunk** (solo si el render no corre).

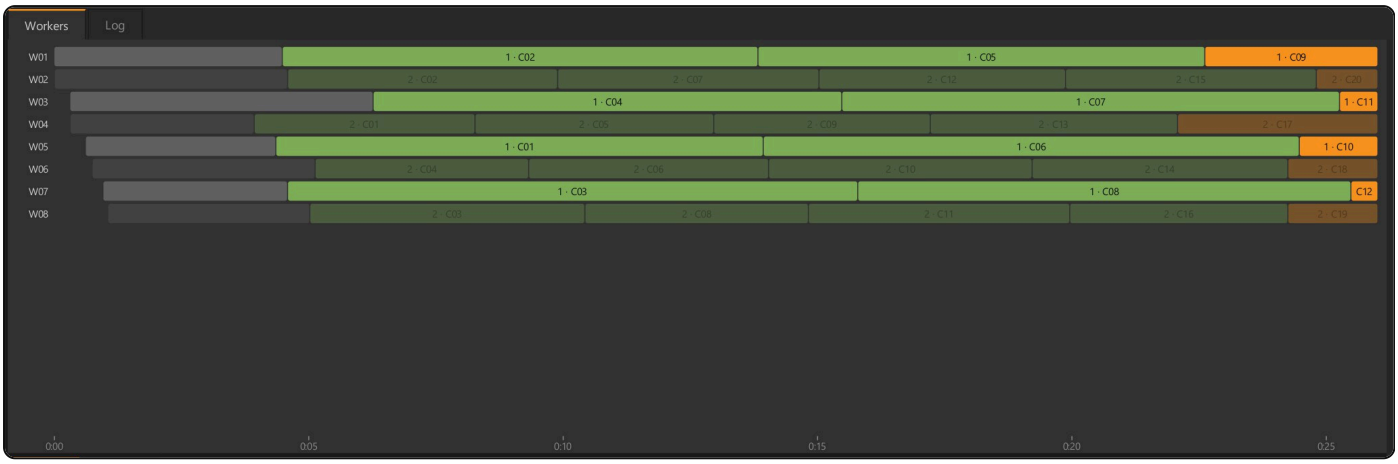
Details



Todo sobre el render en un sitio: el script y su snapshot, los Writes y rutas de salida, formato, fps, rango, versión de Nuke, con qué Processes × Threads × Cache corre, tiempos y las rutas a los logs. Útil cuando necesitas saber exactamente qué se renderizó.

8 Paneles Workers y Log

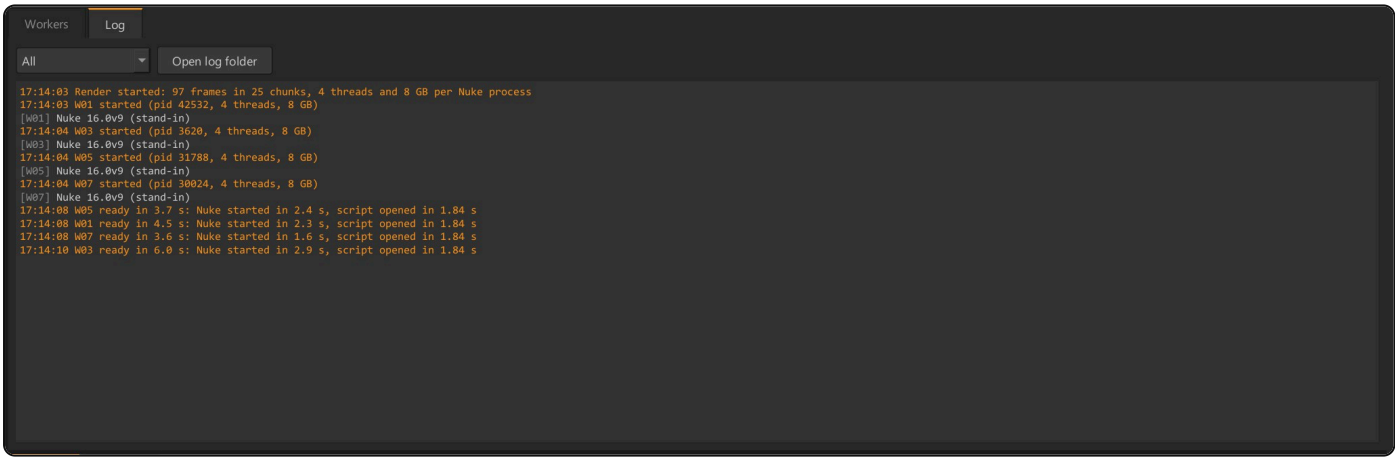
Workers: línea de tiempo de los procesos



Una fila por proceso de Nuke (W01–W08), el tiempo de izquierda a derecha. ⬛ **gris**: Nuke arranca y abre el script (también al cambiarlo por un Nuke nuevo, con la etiqueta *again: reload*). 🟩 **verde**: chunks terminados (con el número de chunk). 🟧 **naranja**: el chunk que se renderiza ahora. Los chunks de otros renders salen atenuados (Settings › *Dim other renders in Workers*). Se ve enseguida si un proceso está parado, vuelve a arrancar o tarda en abrir el script.

Un clic derecho en la fila de un proceso abre el mismo menú que en el panel System (capítulo 9).

Log



Los mensajes de JKrender y la salida de cada proceso de Nuke, con la hora. Filtro: **All**, **JKrender only** (solo mensajes de JKrender), **Errors and warnings**. Open log folder abre la carpeta con los logs de este render: el snapshot del script, `job.json` y el `worker_NN.log` de cada proceso.

> Cada proceso indica en el log cuánto tardó en estar listo, dividido en **arranque de Nuke** y **apertura del script**. Si el arranque tarda 15 s o más, JKrender avisa. Un arranque lento suele ser la licencia o los scripts de inicio del pipeline, no el script en sí.

9 Panel System



Mosaicos y gráficas

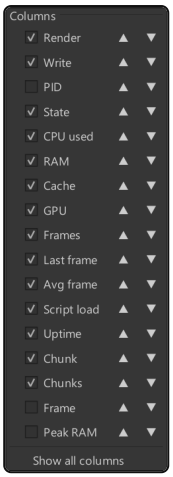
CPU (uso e hilos), **RAM** (total), **Nuke RAM** (lo que usan los procesos de Nuke), **Disk write**, **Disk read**, **Network**, **GPU** (uso y VRAM). La gráfica **Load** muestra CPU, RAM, GPU y VRAM en %, la gráfica **Disk / network** las transferencias en MB/s. **CPU threads**: una barra por hilo de CPU, así se ve si todo el PC está ocupado.

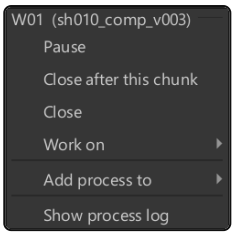
La tabla de procesos

Columna	Significado
Process	Número del proceso (W01…).
Render · Write	En qué render trabaja, y el nombre de su Write.
State	Qué hace: loading, busy, idle, paused, closing… *busy → reload (RAM 91 %)*: tras su chunk recibe un Nuke nuevo, porque la RAM está cerca del límite (capítulo 13). A qué espera, p. ej. *wait: VRAM 75 → 14 % (3 starting)*: ahora 75 %, otro proceso cabe por debajo del 14 %, porque 3 procesos aún arrancan y JKrender les guarda sitio; el tooltip dice más.
CPU used	Cuántos hilos de CPU usa de verdad el proceso.
RAM · Cache	La memoria real del proceso y su caché de Nuke frente al límite.
GPU	Uso de la tarjeta gráfica y VRAM de este proceso.
Frames · Last frame · Avg frame	Cuántos frames hizo, cuánto tardó el último y la media.
Script load	Cuánto tardó Nuke en arrancar y abrir el script.
Uptime · Chunk · Chunks	Cuánto lleva corriendo, en qué chunk está y cuántos hizo.
PID · Frame · Peak RAM	Columnas ocultas: ID del proceso en Windows, frame actual, RAM máxima.

Un clic en la cabecera de la tabla abre la lista de columnas: marcando se muestra u oculta una columna, las flechas cambian el orden. Las columnas también se arrastran por la cabecera. Anchos y orden se guardan al cerrar la ventana.

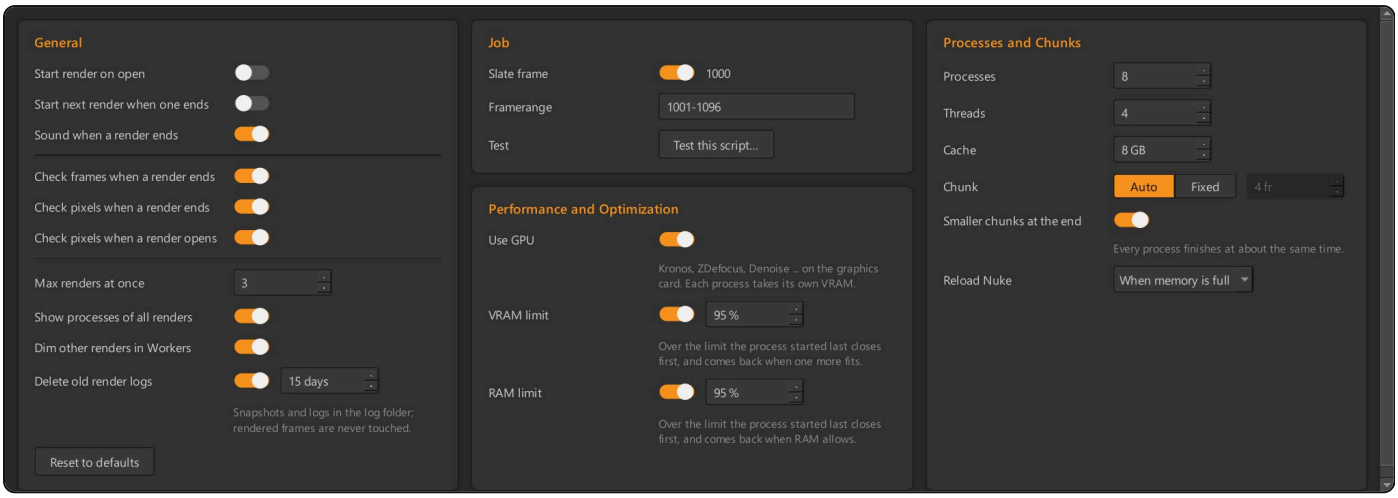
Un clic derecho en un proceso (en la tabla o en una fila de Workers) abre el menú de la derecha. Con varios renders en marcha, *Add process to* tiene un submenú para elegir el render.





Opción del menú	Qué hace
Pause / Resume	Para o deja seguir solo este proceso.
Close after this chunk	El proceso termina su chunk y se cierra, también uno en pausa o uno que ya se cierra por memoria. **Processes** baja en 1 y se queda así.
Close	Cierra el proceso ya. Su chunk vuelve a la cola. Otro puede ocupar su lugar.
Work on › render	Mueve el proceso a otro render en marcha.
Add process	Arranca un proceso más (también con un clic en un hueco vacío de la tabla). **Processes** sube en 1 y se queda así. Los límites de RAM y VRAM siguen valiendo.
Start again · Remove process	En un proceso cerrado: lo arranca otra vez o lo quita (también un sitio vacío donde no corre ningún proceso). Uno quitado no vuelve y **Processes** baja en 1.
Show process log	Abre el log de este proceso.

10 Settings: cada ajuste



Los ajustes se guardan al momento. Cada uno tiene un tooltip. **Reset to defaults** lo devuelve todo a como viene JKrender, también las columnas de las tablas; lo que JKrender aprendió de los scripts se queda.

General

Ajuste	Qué hace	Por defecto
Start render on open	El render arranca en cuanto se abre la ventana, sin pulsar Render.	no
Start next render when one ends	Cuando un render termina o falla, arranca solo el siguiente aún no arrancado, en el orden de las tarjetas. Un render que paraste no arranca nada.	no
Sound when a render ends	Un sonido al terminar el render, cuando acaban las comprobaciones activas. Otro sonido si hay error, frames rotos o un problema en los píxeles. Un render parado no suena.	sí
Check frames when a render ends	Al terminar el render (también tras Stop), una vez cada frame renderizado: existe, no está vacío, está entero y legible, no es mucho más pequeño que el resto. Mientras renderiza, nada lee la carpeta de salida. Desactivado: un frame roto cuenta como hecho hasta que pulses Check frames.	sí
Check pixels when a render ends	Justo tras la comprobación de frames, los píxeles de todos los frames en segundo plano (negros, de un color, iguales, formato distinto; en EXR también NaN e inf). Solo avisa.	sí
Check pixels when a render opens	La misma comprobación de los frames que ya están en el disco, en cuanto se abre un render. Render la detiene.	sí
Max renders at once	**1**: los renders van uno tras otro; el siguiente empieza a cargar mientras el anterior hace sus últimos chunks (el primer plano acaba primero). **2–3**: corren a la vez y se reparten los procesos (no más rápido en total, pero ves pronto frames de cada plano).	3
Show processes of all renders	Workers y System muestran los procesos de todos los renders. Desactivado: solo del elegido.	sí
Dim other renders in Workers	En la línea de tiempo brillan los chunks del render elegido, los demás salen atenuados.	sí
Delete old render logs	Al abrir JKrender borra las carpetas de logs más antiguas que los días indicados. Nunca frames.	sí, 15 días

Unas líneas finas separan los grupos de General. Al renderizar de nuevo, la línea de tiempo de Workers muestra siempre solo la pasada nueva.

Job (para el render elegido arriba)

Ajuste	Qué hace	Por defecto
Slate frame	Activado: cada render renderiza además el frame antes de su Framerange como slate (1001-1020 → slate 1000, 1050-1060 → 1049). Junto al interruptor: el número del slate del render elegido.	sí
Framerange	Los frames del render, p. ej. `1001-1100` o `1001-1010,1050`. Cada render tiene el suyo. Un campo vacío recupera el rango de Nuke. Se confirma con Enter o con un clic en otro sitio. Bloqueado mientras renderiza. Los frames en el disco se quedan.	de Nuke
Test › Test this script…	Mide ajustes para este PC con frames de todo el plano y muestra el más rápido (capítulo 12).	—

Performance and Optimization

Ajuste	Qué hace	Por defecto
Use GPU	Arranca Nuke con `--gpu`, así la tarjeta gráfica calcula los nodos GPU (Kronos, ZDefocus, Denoise…). Un script 4K ligero con Kronos fue 3× más rápido. Cada proceso tiene su propia VRAM.	sí
VRAM limit	Por encima del límite corren menos procesos. Al 97 % se cierra al instante el último proceso arrancado; también si la tarjeta está llena y ningún frame termina en mucho tiempo. A menos de un 5 % del límite ayuda Reload Nuke. Capítulo 13.	sí, 95 %
RAM limit	Por encima del límite corren menos procesos: primero se cierra el último arrancado. Cuando se libera memoria, vuelve. Así Windows nunca empieza a usar swap.	sí, 95 %

Processes and Chunks

Ajuste	Qué hace	Por defecto
Processes	Procesos de Nuke para todos los renders juntos.	8
Threads	Hilos por proceso (`-m`).	4
Cache	Límite de caché de Nuke por proceso en GB (`-c`). Lo que un proceso usa de verdad lo muestra System › RAM. Una caché demasiado pequeña frena el render: Nuke tira y vuelve a calcular.	según RAM
Chunk: Auto / Fixed	**Auto**: unos 4 chunks por proceso, como mucho 10 frames (46 frames con 8 procesos → 2, 96 → 3, 320+ → 10). **Fixed**: el tamaño lo pones a la derecha. Vale para los chunks que aún no han empezado.	Auto
Smaller chunks at the end	Hacia el final del render los chunks se hacen más pequeños hasta frames sueltos, así todos los procesos terminan casi a la vez.	sí
Reload Nuke	Tras su chunk, un proceso cierra Nuke y abre el script en uno nuevo, con la RAM y la VRAM de nuevo al principio. Cuesta una carga del script. **When memory is full**: solo cuando la RAM o la VRAM llevan 30 s seguidos a menos de un 5 % del límite, y de un proceso en uno. **After every chunk**: tras cada chunk. **Never**: nunca. Capítulo 13.	When memory is full

> Processes, Threads y Cache se bloquean mientras corre un render. Para una película muestran 1 proceso con todos los hilos. El número de procesos se cambia en marcha con clic derecho en System (*Add process*, *Close after this chunk*, *Remove process*).

11 Cómo ajustar un render

El camino recomendado para un script nuevo o pesado:

1 **Envía el Write** Ctrl+Alt+R

2 **Create tests** ajustes según el PC

3 **Arranca el test** Round 1 y Round 2

4 **Use these settings** toma la fila verde

5 **Render** y mira System

Los pasos 2–4 están en la ventana Test this script. Los scripts ligeros pueden ir directos con 8 × 4; un test compensa si el render dura decenas de minutos.

Orientación (Ryzen 9950X, 32 hilos, 94 GB RAM, RTX 3090 24 GB)

Tipo de script	Processes × Threads	Por qué	Medido
Comp ligero, keying, grade	**8 × 4**	Un Nuke no usa todos los hilos, más procesos sí.	script de prueba 4K: 8 × 4 el más rápido
Comp pesado (Defocus, Denoise, muchas capas)	**4 × 8** o **2 × 16**	Un proceso usa más hilos. 8 procesos agotan la RAM y se cierran.	comp pesado: 4 × 8 con caché 12 GB 17 fr/min, 2 × 16 16 fr/min
Script cuya VRAM crece (nodos GPU)	**1–2 × 32/16**	La VRAM crece con cada frame; más procesos no caben en la tarjeta.	VRAM +1,7 GB por frame → 1 × 32

Cómo saber que el ajuste no es bueno

Falta memoria

System › RAM está en el límite, State dice *wait: RAM …*, los procesos se cierran y vuelven a arrancar (bloques grises en Workers). **Solución:** menos procesos con más hilos, o menos Cache.

Tarjeta gráfica llena

VRAM al 95 %, los procesos se cierran por VRAM. JKrender propone él mismo menos procesos con más hilos (pero no cambia nada solo). **Solución:** menos procesos con más hilos (Test this script dice cuántos), o desactivar Use GPU.

CPU sin usar

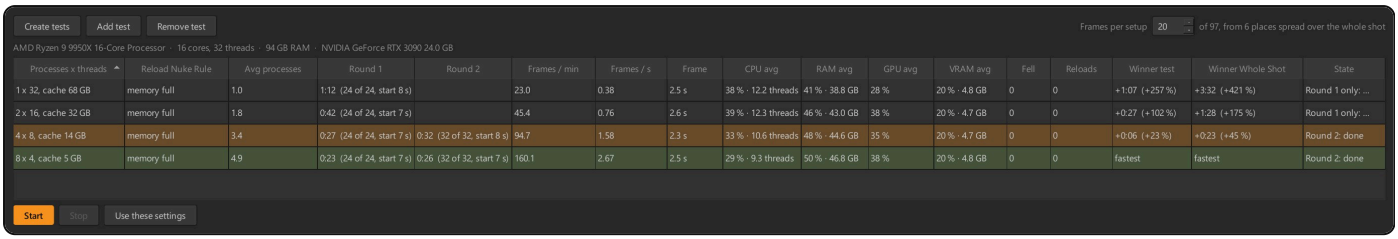
CPU threads está casi vacío y la RAM tiene margen. **Solución:** añade un proceso (clic derecho › Add process) o sube Threads.

Apertura lenta del script

Los bloques grises en Workers son largos, Script load es alto. Al arrancar, los Nukes se pelean por licencias y scripts de inicio. **Solución:** con chunks más largos se nota menos; casi nunca tiene que ver con el script en sí.

> **La mejor medida es el mosaico Speed** (frames por minuto). Compáralo entre ajustes con el mismo script, o deja que lo haga *Test this script*.

12 Test this script



Un test terminado: la fila verde es la más rápida, la naranja la segunda. Winner test muestra cuánto más tardó el test con los demás, Winner Whole Shot lo mismo para todo el plano.

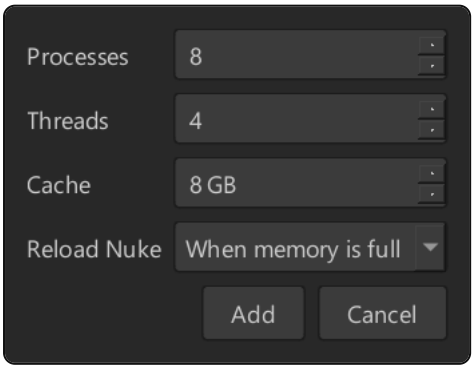
Botones de arriba

Create tests mira el ordenador (la línea bajo los botones: CPU, núcleos e hilos, en Intel núcleos de rendimiento y de eficiencia, RAM, gráfica) y crea **7 ajustes**, desde 1 proceso con todos los hilos hasta tantos procesos como aguante el PC: cada proceso al menos 4 hilos y al menos 4 GB de caché. Con 32 hilos y 94 GB: 1×32, 2×16, 3×10, 4×8, 5×6, 6×5, 8×4. Con más de 64 hilos un proceso recibe como mucho 64 (un grupo de procesadores en Windows). Sustituye todo lo de la tabla.

Add test añade un ajuste propio (Processes, Threads, Cache y Reload Nuke), Remove test quita los elegidos. **Clic derecho** en una fila: **Edit test**. Los tests propios se recuerdan. La regla Reload Nuke de cada ajuste está en la columna **Reload Nuke Rule**, justo después de Processes x threads: *never*, *memory full* o *every chunk*.

La **Cache** de cada ajuste es toda la RAM para Nuke (el límite de RAM menos una reserva para Windows) dividida entre el número de procesos, menos los 4 GB que un proceso usa fuera de su caché. Lo que corra en ese momento no importa.

Frames per setup (arriba a la derecha): cuántos frames renderiza cada ajuste en Round 2. Si el plano tiene menos frames, lo indica y los renderiza todos.



Cómo va el test

Los frames salen de **6 sitios repartidos por todo el plano** (del primer frame al último), unos cuantos seguidos en cada uno. **Todos los ajustes de una ronda renderizan los mismos frames.**

Round 1: cada ajuste un poco, 3 frames por proceso del ajuste con más procesos (con 32 hilos 24 frames), como mucho 3 minutos. **Round 2:** los dos más rápidos (y cualquiera a menos de un 5 % del mejor) otra vez con exactamente Frames per setup frames, pero al menos 4 por proceso (6 procesos → 24).

Winner test: cuánto más tardó el test que con el más rápido (los mismos frames, con el arranque de Nuke). **Winner Whole Shot**: una estimación de todo el plano = el tiempo del test + los frames restantes a Frames / min. El arranque se paga una vez y la velocidad más lenta en cada frame, así que en un plano largo la diferencia crece (p. ej. test +0:10, todo el plano +0:43). El tooltip de la celda muestra ambos tiempos.

Frames / min se cuenta cuando los procesos ya van: el arranque de Nuke y el primer frame de cada proceso no cuentan; en un render largo solo pasan una vez. **Frames / s** es lo mismo por segundo.

Columnas: **Reload Nuke Rule**, **Avg processes** (cuántos procesos renderizaron de media), **Round 1**, **Round 2** (tiempo de la ronda, frames, arranque), **Frames / min**, **Frames / s**, **Frame**, **CPU / RAM / GPU / VRAM avg**, **Fell** (procesos caídos), **Reloads** (cuántas veces se recargó Nuke), **Winner test**, **Winner Whole Shot**, **State**. Un clic en la cabecera ordena; las columnas se pueden ensanchar y mover.

Use these settings pasa la fila verde a Settings, también Reload Nuke. Los frames van a una carpeta de test y se borran; los render hooks del Write no corren en un test. Las tablas quedan en `~/.nuke/JKrender/settings/test_results`.

La ventana **Add test** (y la misma **Edit test** con el clic derecho).

> Durante un test no debería renderizar nada más: mediría dos renders peleándose por el PC. El test no arranca junto a un render.

13 Vigilancia de RAM y VRAM

JKrender vigila la memoria para que Windows no empiece a usar swap y la tarjeta gráfica no se llene.



30 s seguidos a menos de un 5 % del límite
Tras sus chunks los procesos reciben, de uno en uno, un Nuke nuevo (Reload Nuke).

Por encima del límite
Se cierra el último proceso arrancado: ya si aún abre el script, si no tras su chunk.

Vuelta
Vuelven a la vez tantos como quepan; arrancan uno tras otro.

Otra sobrecarga
Espera 5 s, luego 30, 60 y 150 s si se repite en 10 minutos.

Para que un proceso vuelva, tiene que caber bajo el límite un proceso del tamaño que tuvieron los procesos de este render (hasta saberlo: su Cache + 4 GB), con un margen del 2 % y lo que aún van a tomar los procesos en marcha. A qué se espera lo muestra la columna **State** en System, p. ej. *wait: VRAM 75 → 69 %*: la VRAM está al 75 % y tiene que bajar del 69 % para que quepa otro proceso. *wait: VRAM 75 → 14 % (3 starting)*: tres procesos aún arrancan y JKrender les guarda sitio a tamaño completo. Más estados: *wait: 25 s* (un proceso acaba de caer), *wait: W03 1st frame* (espera el primer frame de W03 para ver cuánto usa un proceso), *wait: VRAM ok, checking* (hay sitio, comprueba que se mantiene).

Tarjeta llena y nada avanza: si la VRAM está por encima del límite y ningún proceso termina un frame durante más de 4× el tiempo habitual de frame (al menos un minuto), Nuke probablemente pasó a la CPU y todos los procesos esperan. Entonces se cierra al instante el último proceso arrancado y su chunk vuelve a la cola.

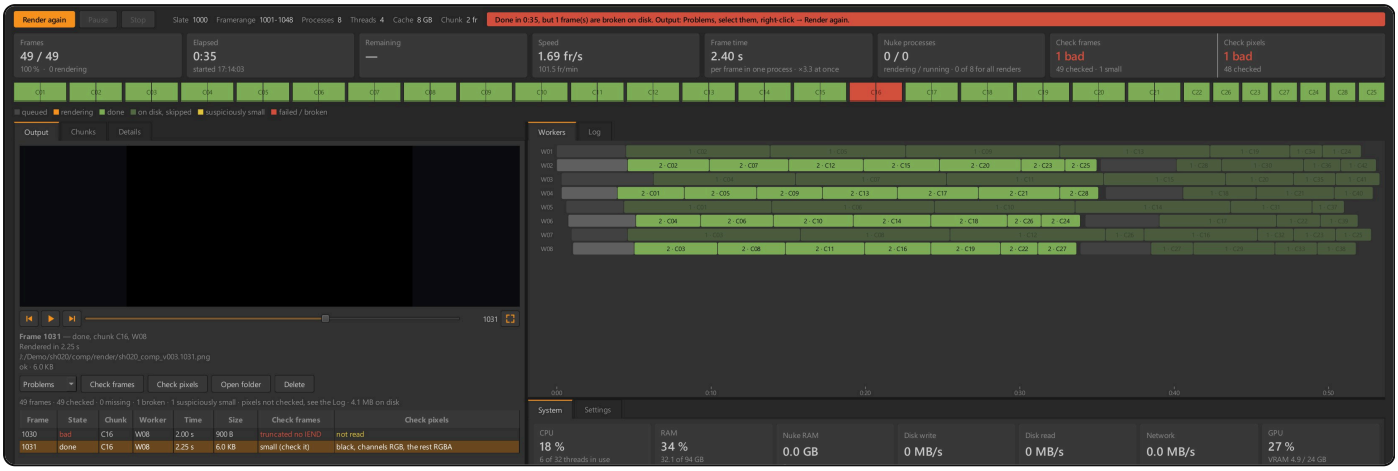
Reload Nuke

Al renderizar, Nuke se queda con memoria que ya no necesita: un proceso de un script pesado puede crecer de 2 a 7 GB de VRAM en sus primeros ~20 frames. Un Nuke nuevo empieza otra vez pequeño, así caben más procesos en la tarjeta. Con el valor por defecto **When memory is full** pasa solo cuando la RAM o la VRAM llevan **30 s seguidos** a menos de un 5 % del límite (un pico suelto no basta), y **de un proceso en uno**: el siguiente solo cuando el Nuke nuevo ha renderizado su primer frame y la memoria sigue cerca del límite. Los demás siguen renderizando mientras tanto. Un proceso que recibe un Nuke nuevo muestra en State *busy → reload (VRAM 93 %)*; en los últimos chunks no se cambia nada.

No existe un tope de VRAM por proceso. Cache (`-c`) limita solo la caché de RAM de Nuke. La memoria gráfica de un proceso no se puede limitar: Windows no sabe hacerlo y los ajustes de caché Blink de Nuke no tienen efecto en un render (medido). Por eso JKrender controla la VRAM con el número de procesos y con Nukes nuevos. Cache es un límite de caché, no toda la memoria de un proceso; la cifra real está en System › RAM.

14 Comprobaciones, sonidos, caídas y datos

Comprobación de la salida



Un render con un frame cortado (1030) y un frame negro (1031), ambos encontrados a mano con Check frames y Check pixels: barra roja, frames rojos en el mapa, el filtro Problems muestra solo esos.

Comprobación	Cuándo	Qué busca
Check frames	Al terminar el render y tras Stop (se puede desactivar), y con el botón. Siempre al abrir un render y antes de Render.	Archivos que faltan, vacíos, cortados e ilegibles (EXR, PNG, JPEG, DPX), frames sospechosamente pequeños.
Check pixels	Al terminar el render, al abrirlo (se puede desactivar) y con el botón.	EXR, PNG, JPEG, TIFF y DPX: frames negros, vacíos y de un solo color, frames iguales a otro, tamaño, canales y profundidad de bits distintos. En EXR también NaN, inf y compresión. Otros formatos: solo archivos idénticos.
Movie check	En MOV, MP4 y MXF.	Códec, perfil, fps, resolución, timecode y número de frames frente a los ajustes del Write.

Sonido tras el render

Tras el render las comprobaciones van en el orden **Check frames** → **Check pixels**; el sonido llega cuando acaban las que están activas (con las dos desactivadas, justo tras el render).

Sonido alegre: todo bien. **Sonido de error**: el render falló, tiene frames rotos o la comprobación de píxeles encontró un problema. Un render parado a mano no suena.

Cuando JKrender se cae

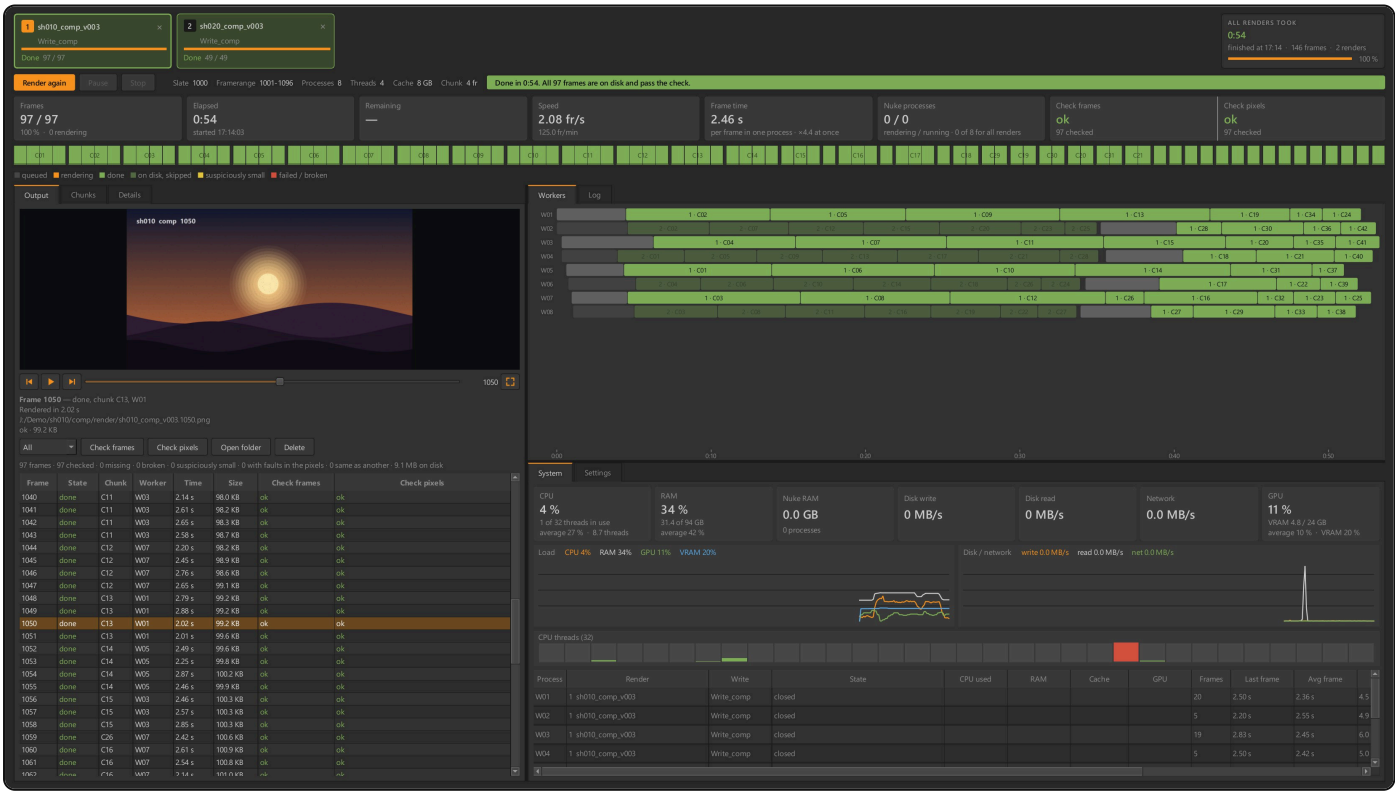
Las tarjetas vuelven la próxima vez que se abre JKrender (por ejemplo con el siguiente render enviado desde Nuke). Lo que hay en el disco cuenta y los procesos de Nuke sobrantes se cierran. Un render que estaba en marcha espera a Render y luego renderiza lo que falta. Si cierras la ventana normalmente, no vuelve nada.

Datos de render

Cada render escribe `report.json` en su carpeta de log: CPU, RAM y tarjeta gráfica de este PC, ajustes, velocidad, picos de memoria, cierres por límites, arranque de Nuke, apertura del script y los tipos de nodos del script. **No contiene nombres de scripts, rutas ni errores**; un script solo se reconoce por un ID que no cambia entre sus versiones. Si encuentras un fallo, envía la carpeta de log del render.

15 Ratón y teclas: chuleta

Dónde y qué	Resultado
`Ctrl+Alt+R` en Nuke	Envía los Writes elegidos a JKrender.
Clic en una tarjeta de render	Elige el render; toda la ventana muestra sus datos.
`Shift` + clic en tarjetas	Elige varios renders para Render, Render again, Pause, Stop.
Clic en un frame del mapa de frames	Abre el frame en la vista previa.
`Space`	Reproducir / parar la vista previa (nunca para el render).
Rueda / arrastrar en la vista previa	Zoom / mover la imagen.
Doble clic en la vista previa	Frame entero; si ya lo está, vista grande.
`Esc`	Volver de la vista grande.
Clic derecho en un frame (Output)	Render this frame / batch, Delete this frame / batch.
Clic derecho en un chunk (Chunks)	Render again, Show process log, Delete frames from chunk.
Clic derecho en un proceso (System, Workers)	Pause, Close after this chunk, Close, Remove process, Work on, Add process, Start again, Show process log.
Clic derecho en un hueco vacío de System	Add process, Remove process.
Clic en la cabecera de la tabla System	Qué columnas y en qué orden.
Arrastrar el borde de un panel	Cambia el tamaño de los paneles (se guarda).
Framerange + `Enter`	Confirma un rango nuevo (Settings › Job).



Un render terminado: barra verde; la tarjeta de arriba a la derecha muestra cuánto duraron los renders.

16 Cuando algo no va

Problema	Qué hacer
Los procesos de Nuke tardan en arrancar (30–50 s)	Suele ser el arranque de Nuke (licencia, scripts de inicio del pipeline) mientras el PC renderiza. El Log lo desglosa: *Nuke started in …* / *script opened in …*.
Los procesos se cierran y vuelven a arrancar	Se acabó la RAM o la VRAM (State en System, Log). Menos procesos con más hilos o menos Cache; Test this script muestra qué es lo más rápido.
State dice *busy* → *reload*	La RAM o la VRAM llevan 30 s cerca del límite; tras su chunk el proceso recibe un Nuke nuevo (Settings › Reload Nuke). Si pasa todo el rato, corren más procesos de los que caben.
Todo el render va más lento que la última vez	Revisa la Cache: demasiado pequeña (p. ej. 4 GB para un script pesado) significa que Nuke tira y vuelve a calcular, y los frames tardan varias veces más.
El script se abre en Nuke con un error de knob	P. ej. *LensDistortion2.serialiseKnob: Failed to load* (un nodo de otra versión, un plug-in que falta). JKrender sigue renderizando, como Nuke en la interfaz; el aviso está en el log del proceso.
Barra amarilla «not checked»	La comprobación de frames está desactivada (Check frames when a render ends) o no se hizo. Pulsa Check frames.
Tarjeta o barra roja	El tooltip de la tarjeta dice qué falló; Output › el filtro Problems muestra qué frames. Clic derecho *Render this frame / batch*. La barra se recalcula también tras Check frames y Check pixels a mano.
No hay vista previa EXR	Faltan numpy/OpenEXR. Borra `lib/DECLINED` y reinicia Nuke, o instálalos según el README.
El render no arrancó enseguida	Estado Waiting: otros renders ya usan el PC (Max renders at once), o espera memoria.
Vaciar la caché de Nuke no funciona	Tras actualizar JKrender, reinicia Nuke una vez.
No sé qué se renderizó	El panel Details y Log › Open log folder (snapshot del script y job.json).

> **¿Has encontrado un fallo?** Envía al autor la carpeta de log del render (Open Renders Folder) y una descripción de lo que pasó.